Project Work[SQL]

SECTION-1

1) Find total number of transactions.
```
SELECT COUNT(*)AS TOTAL_TRANSACTIONS FROM SALES;
```

2) Find total revenue generated.
```
SELECT SUM(amount)AS TOTAL_REVENUE FROM SALES;
```

3) Find total unique customers.
```
SELECT COUNT(DISTINCT CUSTOMER_ID)AS UNQ_CUST FROM SALES;
```

4) Find average order value.
```
SELECT AVG(AMOUNT)AS AVG_ORDERS FROM SALES;
```

5) Find total quantity sold.
```
SELECT SUM(QUANTITY)AS TOTAL_QUANT FROM SALES;
```

6) Find minimum and maximum sale amount.
```
SELECT MIN(AMOUNT)AS MIN_AMT,MAX(AMOUNT)AS MAX_AMT FROM SALES;
```

7) Find average discount given.
```
SELECT AVG(discount)AS AVG_DISCOUNT FROM SALES;
```

8) Find total sales per store.
```
SELECT S.STORE_NAME,SUM(SA.AMOUNT)AS TOTAL_AMT FROM SALES SA
JOIN STORES S ON SA.store_id = S.store_id
GROUP BY S.STORE_NAME;
```

9) Find total sales per category.
```
SELECT P.category,
SUM(SA.AMOUNT)AS TOTAL_QUANT_AMOUNT FROM SALES SA
JOIN products P ON SA.product_id = P.product_id
GROUP BY P.category;
```

10) Find number of transactions per day.
```
SELECT SALE_DATE,COUNT(*)AS NO_TRANS FROM sales
GROUP BY SALE_DATE ORDER BY SALE_DATE;
```

SECTION -2

11) Replace NULL discount with 0.
```
SELECT NVL(discount,0)AS DISCOUNT_VAL FROM sales;
```

12) Find number of NULL discounts.
```
SELECT COUNT(*) AS NO_DISCOUNT FROM SALES WHERE DISCOUNT IS NULL;
```

13) Calculate net revenue (amount - discount).
```
SELECT AMOUNT - NVL(DISCOUNT,0)AS REVENUE FROM SALES;
```

14) Replace NULL quantity with 1.

```
SELECT NVL(QUANTITY,1)AS QUANTITY FROM SALES;
```

15) Identify rows where amount is NULL.

```
SELECT * FROM SALES WHERE AMOUNT IS NULL;
```

[NO NULL VALUES]

16) Use COALESCE to handle multiple NULL columns.

```
SELECT COALESCE(DISCOUNT,0,AMOUNT,0)AS CLEAN_VAL FROM SALES;
```

17) Create a cleaned column for revenue.

```
SELECT SALE_ID,
AMOUNT,NVL(DISCOUNT,0)AS DISCOUNT,
AMOUNT - NVL(DISCOUNT,0)AS CLEAN_REVENUE
FROM SALES;
```

18) Check percentage of missing data.

```
SELECT
(COUNT(CASE WHEN DISCOUNT IS NULL THEN 1 END)* 100)/COUNT(*)AS PER_NULL_DISCOUNT
FROM SALES;
```

19) Flag rows with missing values.

```
SELECT *,
CASE WHEN AMOUNT IS NULL OR DISCOUNT IS NULL OR QUANTITY IS NULL
THEN 'MISSING'
ELSE 'good'
END AS FLAG_ROWS
FROM SALES;
```

20) Prepare dataset for analysis.

[IGNORED]

SECTION 3

21) Find sales in last 30 days.

```
SELECT * FROM sales WHERE sale_date
BETWEEN DATE '2025-03-01' AND DATE '2025-03-31';
```

22) Find sales above 5000.

```
SELECT * FROM sales WHERE AMOUNT > 5000;
```

23) Find sales between 1000 and 5000.

```
SELECT * FROM SALES WHERE AMOUNT BETWEEN 1000 AND 5000;
```

24) Find sales from CMR store.

```
SELECT S.STORE_NAME,SA.store_id,SA.QUANTITY,SA.AMOUNT FROM SALES SA
JOIN STORES S ON SA.store_id = S.store_id
WHERE STORE_NAME = 'CMR';
```

25) Find Electronics category sales.

```
SELECT P.category,SA.amount FROM  SALES SA
JOIN products P ON SA.product_id = P.product_id
WHERE P.category = 'Electronics';
```

26) Find sales excluding Grocery category.

```
SELECT P.category,SA.amount FROM  SALES SA
JOIN products P ON SA.product_id = P.product_id
WHERE P.category <> 'Grocery';
```

27) Find customers from specific store.

```
select C.customer_name
FROM SALES SA
JOIN CUSTOMERS C ON SA.CUSTOMER_ID = C.CUSTOMER_ID
JOIN STORES S ON S.STORE_ID = SA.STORE_ID
WHERE S.STORE_NAME = 'SR'
```

28) Find high discount transactions.

```
SELECT MAX(DISCOUNT)AS HIGH_DISCOUNT FROM sales
```

29) Find low quantity sales.

```
SELECT MIN(QUANTITY)AS LOW_QUANTITY FROM sales
```

30) Find recent transactions.

```
SELECT * FROM SALES ORDER BY SALE_DATE DESC;
```

SECTION 4

31) Revenue per store.

```
SELECT S.STORE_NAME,
SUM(SA.AMOUNT) AS TOTAL_REV_STORE
FROM SALES SA
JOIN STORES S ON SA.STORE_ID = S.STORE_ID
GROUP BY S.STORE_NAME;
```

32) Revenue per category.

```
SELECT P.CATEGORY,
SUM(SA.AMOUNT) AS TOTAL  REV  STORE
FROM SALES SA
JOIN products P ON SA.PRODUCT  ID = P.PRODUCT  ID
GROUP BY P.CATEGORY;
```

33) Revenue per customer.

```
SELECT C.CUSTOMER  NAME,
SUM(SA.AMOUNT) AS TOTAL_REV_STORE
FROM SALES SA
JOIN CUSTOMERS C ON SA.CUSTOMER_ID = C.CUSTOMER_ID
GROUP BY C.CUSTOMER_NAME;
```

34) Monthly revenue.

```
SELECT EXTRACT(MONTH FROM sale  date)AS MONTH,
SUM(amount)AS REVENUE FROM sales SA
GROUP BY EXTRACT(MONTH FROM sale_date);
```

35) Daily revenue.

```
SELECT EXTRACT(DAY FROM sale_date)DAYS,
SUM(amount)REVENUE_FOR_DAYS FROM sales SA
GROUP BY EXTRACT(DAY FROM sale_date);
```

36) Average sales per store.

```
SELECT S.STORE_NAME,
AVG(SA.AMOUNT) AS AVG_REV_STORE
FROM SALES SA
JOIN STORES S ON SA.STORE_ID = S.STORE_ID
GROUP BY S.STORE_NAME;
```

37) Total quantity per category.

```
SELECT P.CATEGORY,
SUM(SA.QUANTITY) AS TOTAL_QUANTITY
FROM SALES SA
JOIN products P ON SA.PRODUCT_ID = P.PRODUCT_ID
GROUP BY P.CATEGORY;
```

38) Total discount per store.

```
SELECT S.STORE_NAME,
SUM(NVL(SA.DISCOUNT,0)) AS TOTAL_DISCOUNT
FROM SALES SA
JOIN STORES S ON SA.STORE_ID = S.STORE_ID
GROUP BY S.STORE_NAME;
```

39) Customer-wise total spending.

```
SELECT C.CUSTOMER_NAME,
SUM(SA.AMOUNT) AS TOTAL_SPENDING
FROM SALES SA
JOIN CUSTOMERS C ON SA.CUSTOMER_ID = C.CUSTOMER_ID
GROUP BY C.CUSTOMER_NAME;
```

40) Store-wise transaction count.

```
SELECT S.STORE_NAME,S.STORE_ID,COUNT(*)AS TOTAL_TRANSATIONS
FROM SALES SA
JOIN STORES S ON S.STORE_ID = SA.STORE_ID
GROUP BY S.STORE_NAME,S.STORE_ID;
```

SECTION 5

41) Customers with revenue > 20000.

```
SELECT C.CUSTOMER_NAME,SUM(SA.AMOUNT)AS REVENUE
FROM SALES SA
JOIN CUSTOMERS C ON SA.CUSTOMER_ID = C.CUSTOMER_ID
GROUP BY C.CUSTOMER_NAME
HAVING SUM(SA.AMOUNT) > 20000;
```

42) Stores with revenue > 100000.

```
SELECT S.STORE_NAME,SUM(SA.AMOUNT)AS REVENUE
FROM SALES SA
JOIN STORES S ON SA.STORE_ID = S.STORE_ID
GROUP BY S.STORE_NAME
HAVING SUM(SA.AMOUNT) > 100000;
```

43) Categories with avg sales > 3000.

```
SELECT P.CATEGORY,AVG(SA.AMOUNT)AS AVG_SALES
FROM SALES SA
JOIN PRODUCTS P ON SA.PRODUCT_ID = P.PRODUCT_ID
GROUP BY P.CATEGORY
HAVING AVG(SA.AMOUNT) > 3000;
```

44) Customers with more than 5 transactions.

```
select CUSTOMER_ID,COUNT(*)AS TOTAL_TARNS
FROM SALES
GROUP BY CUSTOMER_ID
HAVING COUNT(*)> 5;
```

45) Stores with high discount usage.

```
SELECT S.STORE_NAME,AVG(SA.DISCOUNT)AS AVG_DIS
FROM SALES SA
JOIN STORES S ON S.STORE_ID = S.STORE_ID
GROUP BY S.STORE_NAME
HAVING AVG(SA.DISCOUNT) >100;
```

46) Categories with low performance.

```
SELECT P.CATEGORY,SUM(SA.amount)AS TOTAL_REVENUE
FROM SALES SA
JOIN PRODUCTS P ON SA.PRODUCT_ID = P.PRODUCT_ID
GROUP BY P.CATEGORY
HAVING SUM(SA.AMOUNT) < 5000000;
```

47) Customers with high avg order value.

```
SELECT CUSTOMER_ID,AVG(AMOUNT)AS AVG_ORDER
FROM SALES GROUP BY CUSTOMER_ID
HAVING AVG(AMOUNT) >3000;
```

48) Find top 5 customers using HAVING.

```
SELECT * FROM (
SELECT CUSTOMER_ID,SUM(AMOUNT)AS TOTAL_SPENT
FROM sales GROUP BY CUSTOMER_ID
HAVING SUM(AMOUNT) > 10000
ORDER BY TOTAL_SPENT DESC)
WHERE ROWNUM <=5;
```

49) Stores with max transactions.

```
SELECT * FROM(SELECT STORE_ID,COUNT(SALE_ID)AS TRANS_COUNT
FROM SALES GROUP BY STORE_ID
ORDER BY TRANS_COUNT)WHERE ROWNUM =1;
```

50) Categories with high growth.

```
SELECT P.CATEGORY,SUM(SA.amount)AS TOTAL_REVENUE
FROM SALES SA
JOIN PRODUCTS P ON SA.PRODUCT_ID = P.PRODUCT_ID
GROUP BY P.CATEGORY
HAVING SUM(SA.AMOUNT) > 5000000;
```

SECTION 6

51) Classify customers as High/Medium/Low spenders.

```
SELECT CUSTOMER_ID,
SUM(AMOUNT)AS TOTAL_AMT,
CASE WHEN SUM(AMOUNT)>200000 THEN 'HIGH'
WHEN SUM(AMOUNT) BETWEEN 100000 AND 200000 THEN 'MID'
 ELSE 'LOW'
 END AS SPENDER
 FROM SALES
GROUP BY CUSTOMER_ID;
```

52) Classify transactions as Big/Small.

```
SELECT SALE_ID,
CASE WHEN AMOUNT > 5000 THEN 'BIG'
ELSE 'SMALL'
END AS TRANS_TYPE
FROM SALES;
```

53) Create discount flag.

```
SELECT DISCOUNT,
CASE WHEN DISCOUNT IS NULL THEN 'NO'
ELSE 'YES' END AS DISCOUNT FROM SALES;
```

54) Segment stores based on revenue.

```
SELECT STORE_ID,SUM(AMOUNT) AS REVENUE,
CASE WHEN SUM(AMOUNT) > 500000 THEN 'HIGH'
WHEN SUM(AMOUNT) BETWEEN 200000 AND 500000 THEN 'MEDIUM'
ELSE 'LOW'
END AS SEGMENT
FROM SALES
GROUP BY STORE_ID;
```

55) Categorize customers based on frequency.

```
SELECT CUSTOMER_ID,
COUNT(*) AS TOTAL_TRANS,
CASE WHEN COUNT(*) > 10 THEN 'FREQ'
ELSE 'RARE' END AS CUST_TYPE
FROM sales GROUP BY CUSTOMER_ID;
```

56) Create sales buckets.

```
SELECT SALE  ID,
CASE WHEN AMOUNT < 1000 THEN 'LOW'
WHEN AMOUNT BETWEEN 1000 AND 5000 THEN 'MEDIUM'
ELSE 'HIGH' END AS SALE  VAL
FROM SALES;
```

57) Identify premium customers.

```
SELECT CUSTOMER  ID,SUM(AMOUNT)AS TOTAL  SPENT,
CASE WHEN SUM(AMOUNT) > 300000 THEN 'PREMIUM'
ELSE 'NORMAL'
END AS CUST_STATE FROM SALES GROUP BY CUSTOMER_ID;
```

58) Mark high-value transactions.

```
SELECT CUSTOMER_ID,
CASE WHEN AMOUNT > 8000 THEN 'HIGH'
ELSE 'LOW'
END AS HIGH_VALUE_TRANS FROM SALES;
```

59) Classify categories based on performance.

```
SELECT P.CATEGORY,SUM(SA.AMOUNT)AS CATEGORY  REV,
CASE WHEN (SA.AMOUNT) BETWEEN 5000 AND 10000 THEN 'AVG  PERFORMANCE'
ELSE 'LOW_PERFOMANCE' END AS PERFORMANCE FROM SALES SA JOIN PRODUCTS P
ON SA.PRODUCT_ID = P.PRODUCT_ID GROUP BY P.CATEGORY,SA.AMOUNT;
```

60) Create custom labels for reporting.

```
SELECT SALE  ID,AMOUNT,
CASE WHEN AMOUNT > 8000 THEN 'HIGH'
WHEN AMOUNT BETWEEN 4000 AND 8000 THEN 'MID'
ELSE'LOW' END AS LABLE
FROM SALES;
```

SECTION 7

61) Join sales with customers table.

```
SELECT * FROM SALES SA
JOIN CUSTOMERS C
ON SA.CUSTOMER_ID= C.CUSTOMER_ID;
```

62) Join sales with stores table.

```
SELECT * FROM SALES SA
JOIN STORES S
ON SA.STORE_ID = S.STORE_ID
```

63) Join sales with products table.

```
SELECT * FROM SALES SA
JOIN PRODUCTS P
ON SA.PRODUCT_ID = P.PRODUCT_ID;
```

64) Find customer name with store name and sales.

```
SELECT C.CUSTOMER_NAME,
S.STORE_NAME,
SA.AMOUNT
FROM sales SA
JOIN CUSTOMERS C ON C.CUSTOMER_ID = SA.CUSTOMER_ID
JOIN STORES S ON S.STORE_ID = SA.STORE_ID;
```

65) Find category-wise revenue using joins.

```
SELECT P.CATEGORY,
SUM(SA.AMOUNT) AS REVENUE
FROM SALES SA JOIN PRODUCTS P
ON SA.PRODUCT_ID = P.PRODUCT_ID
GROUP BY P.CATEGORY;
```

66) Find store-wise customer count.

```
SELECT S.STORE_NAME,
COUNT(DISTINCT SA.CUSTOMER_ID)AS CUST_COUNT
FROM SALES SA
JOIN STORES S ON SA.STORE_ID = S.STORE_ID
GROUP BY S.STORE_NAME;
```

67) Find top product per store.

68) Find customers who purchased from multiple stores.

```
SELECT CUSTOMER_ID FROM SALES
GROUP BY CUSTOMER_ID HAVING COUNT(DISTINCT STORE_ID) > 1;
```

69) Find sales with product category.

```
SELECT P.CATEGORY,
SA.SALE_ID,SA.AMOUNT FROM SALES SA
JOIN PRODUCTS P ON SA.PRODUCT_ID = P.PRODUCT_ID;
```

70) Find revenue per customer per store.

```
SELECT C.CUSTOMER_NAME,
S.STORE_NAME,
SUM(SA.AMOUNT) AS REVENUE FROM SALES SA
JOIN CUSTOMERS C ON SA.CUSTOMER_ID = C.CUSTOMER_ID
JOIN STORES S ON SA.STORE_ID = S.STORE_ID
GROUP BY C.CUSTOMER_NAME,S.STORE_NAME;
```

71) Find customers with no transactions (LEFT JOIN).

```
SELECT C.CUSTOMER_NAME
FROM CUSTOMERS C
LEFT JOIN SALES SA ON C.CUSTOMER_ID = SA.CUSTOMER_ID
WHERE SA.CUSTOMER_ID IS NULL;
```

72) Find missing product mappings.

```
SELECT * FROM SALES SA
LEFT JOIN PRODUCTS P
ON SA.PRODUCT_ID = P.PRODUCT_ID
WHERE SA.PRODUCT_ID IS NULL;
```

73) Find store performance using joins.

```
SELECT S.STORE_NAME,
SUM(SA.AMOUNT)AS REVENUE,
COUNT(*) AS TOTAL_TRANS
FROM SALES SA JOIN STORES S
ON SA.STORE_ID = S.STORE_ID
GROUP BY S.STORE_NAME;
```

74) Find cross-category purchases.

```
SELECT SA.CUSTOMER_ID FROM SALES SA
JOIN PRODUCTS P ON SA.PRODUCT_ID = P.PRODUCT_ID
GROUP BY SA.CUSTOMER_ID
HAVING COUNT(DISTINCT P.CATEGORY) >1;
```

75) Find total revenue using joins.

```
SELECT SUM(SA.amount)AS TOTAL_REV
FROM SALES SA JOIN STORES S ON SA.STORE_ID = S.STORE_ID;
```

SECTION 8

76) Calculate total spend per customer using window function.

```
SELECT CUSTOMER_ID,AMOUNT,
SUM(AMOUNT) OVER(PARTITION BY CUSTOMER_ID)AS TOTAL
FROM SALES;
```

77) Rank customers by revenue.

```
SELECT CUSTOMER_ID,SUM(AMOUNT)AS AMT,
RANK() OVER (ORDER BY SUM(AMOUNT) DESC) AS RANKING
FROM SALES GROUP BY CUSTOMER_ID;
```

78) Find top customer per store.

```
SELECT STORE_ID,CUSTOMER_ID,SUM(AMOUNT) AS AMOUNT,
RANK() OVER(PARTITION BY STORE_ID ORDER BY SUM(AMOUNT) DESC)AS RANKS
FROM SALES GROUP BY STORE_ID,CUSTOMER_ID;
```

79) Calculate running total of sales.

```
SELECT
SALE_DATE,AMOUNT,SUM(AMOUNT) OVER(ORDER BY sale_date)AS RUNNING_TOTAL
FROM SALES;
```

80) Find previous sale using LAG.

```
SELECT SALE  ID,AMOUNT,
LAG(AMOUNT)OVER (ORDER BY SALE_DATE)AS PREV_DAY FROM SALES;
```

81) Find next sale using LEAD.

```
SELECT SALE_ID,AMOUNT,
LEAD(AMOUNT)OVER (ORDER BY SALE_DATE)AS NXT_DAY FROM SALES;
```

82) Calculate sales growth.

```
SELECT SALE  DATE,AMOUNT,
AMOUNT - LAG(AMOUNT)OVER(ORDER BY SALE_ID)AS GROWTH FROM SALES;
```

83) Find difference between transactions.

```
SELECT SALE_DATE,AMOUNT,
AMOUNT - LAG(AMOUNT)OVER(ORDER BY SALE_ID)AS DIFF FROM SALES;
```

84) Calculate cumulative revenue.

```
SELECT SALE_DATE,SUM(AMOUNT) OVER(ORDER BY SALE_DATE
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS CUMM_REV
FROM SALES;
```

85) Find moving average of sales.

```
SELECT SALE  DATE,AMOUNT,AVG(AMOUNT)OVER(ORDER BY SALE  DATE
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)AS MOVING  AVG
FROM SALES;
```

86) Rank stores by revenue.

```
SELECT STORE  ID,SUM(AMOUNT) AS REVENUE,
RANK() OVER(ORDER BY SUM(AMOUNT) DESC)AS RANKS FROM SALES
GROUP BY STORE_ID;
```

87) Find second highest sale.

```
SELECT STORE  ID,SUM(AMOUNT) AS REVENUE,
RANK() OVER(ORDER BY SUM(AMOUNT) DESC)AS RANKS FROM SALES
GROUP BY STORE_ID;
```

88) Partition data by store and analyze.

```
SELECT STORE_ID,AMOUNT,
SUM(AMOUNT) OVER(PARTITION BY store_id)AS STORE_SALE FROM SALES;
```

89) Find customer ranking per store.

```
SELECT STORE  ID,CUSTOMER  ID,SUM(AMOUNT)as revenue,
RANK() OVER(PARTITION BY  STORE  ID ORDER BY SUM(AMOUNT) DESC)
AS RANKING
FROM SALES GROUP BY STORE_ID,CUSTOMER_ID;
```

90) Calculate frequency using window.

```
SELECT CUSTOMER_ID,COUNT(*) OVER(PARTITION BY CUSTOMER_ID)
AS FREQUENCIES FROM SALES;
```

SECTION -9

91) Divide customers into 4 segments.

```
SELECT CUSTOMER  ID,AMOUNT,
NTILE(4) OVER(ORDER BY AMOUNT DESC) AS SEGMENT FROM sales
GROUP BY CUSTOMER_ID,AMOUNT;
```

92) Identify top 25% customers.

```
SELECT *
FROM (
SELECT
 customer_id,
amount,
NTILE(4) OVER (ORDER BY amount DESC) AS top  25
FROM sales
) t
WHERE top_25 = 1;
```

93) Identify bottom 25% customers.

```
SELECT *
FROM (
SELECT
customer  id,
amount,
NTILE(4) OVER (ORDER BY amount DESC) AS top_25
FROM sales
) t
WHERE top_25 = 4;
```

94) Segment stores into 3 groups.

```
SELECT  STORE  ID,amount,
NTILE(3) OVER(ORDER BY STORE_ID)AS SEGMA FROM SALES ORDER BY STORE_ID,amount;
```

95) Create quartiles based on revenue.

```
SELECT CUSTOMER  ID,SUM(AMOUNT)AS REVENUE,
NTILE(4) OVER(ORDER BY SUM(AMOUNT) DESC) AS SEGMENT  FROM sales GROUP BY
CUSTOMER_ID;
```

96) Segment customers per store.

```
SELECT CUSTOMER  ID,STORE  ID,AMOUNT,NTILE(4)
OVER(PARTITION BY STORE  ID ORDER BY AMOUNT DESC)AS SEGMENT
FROM SALES GROUP BY CUSTOMER_ID,STORE_ID,AMOUNT;
```

97) Identify premium segment.

```
SELECT *
FROM (
 SELECT
 customer  id,
 amount,
 NTILE(4) OVER (ORDER BY amount DESC) AS PREMIUM
FROM sales
) t
WHERE PREMIUM = 1;
```

98) Find mid-level customers.

```
SELECT *
FROM (
 SELECT
 customer  id,
amount,
NTILE(4) OVER (ORDER BY amount DESC) AS SEGMENT
FROM sales
) t
WHERE SEGMENT IN (2,3);
```

99) Analyze bucket distribution.

```
SELECT BUCKETS,
COUNT(*)AS CUST  COUNT,MAX(AMOUNT)AS MAX  AMT,MIN(AMOUNT)AS MIN  AMT
FROM(SELECT NTILE(4) OVER (ORDER BY  AMOUNT DESC)AS BUCKETS,AMOUNT FROM SALES)
GROUP BY BUCKETS ORDER BY  BUCKETS;
```

100) Create marketing segments.

```
SELECT CUSTOMER  ID,AMOUNT,
CASE WHEN NTILE(4) OVER(ORDER BY AMOUNT DESC) = 1 THEN 'PLATINIUM'
WHEN NTILE(4) OVER(ORDER BY AMOUNT DESC) = 2 THEN 'GOLD'
WHEN NTILE(4) OVER(ORDER BY AMOUNT DESC) = 3 THEN 'SILVER'
ELSE 'BRONZE' END AS SEGMENT FROM SALES GROUP BY CUSTOMER_ID,AMOUNT;
```

SECTION 10

101) Find highest sale.

```
SELECT DISTINCT FIRST_VALUE(AMOUNT)OVER(ORDER BY AMOUNT DESC )AS HIGH_VALUE
FROM SALES;
```

102) Find lowest sale.

```
SELECT DISTINCT FIRST  VALUE(AMOUNT)OVER(ORDER BY AMOUNT )AS LOW  VALUE FROM
SALES;
```

103) Compare each sale with highest.

```
SELECT SALE_ID,AMOUNT,FIRST_VALUE(AMOUNT)OVER(ORDER BY AMOUNT DESC)AS
HIGHEST_VALUE,
AMOUNT - FIRST_VALUE(AMOUNT)OVER(ORDER BY AMOUNT DESC) AS DIFF FROM SALES;
```

104) Compare each sale with lowest.

```
SELECT SALE_ID,AMOUNT,FIRST_VALUE(AMOUNT)OVER(ORDER BY AMOUNT)AS
LOWEST_VALUE,
AMOUNT - FIRST_VALUE(AMOUNT)OVER(ORDER BY AMOUNT ) AS DIFF FROM SALES;
```

105) Find highest sale per store.

```
SELECT STORE_ID,SALE_ID,AMOUNT,FIRST_VALUE(AMOUNT)OVER(PARTITION BY STORE_ID
ORDER BY AMOUNT DESC)AS HIGH_SALE_PER_STORE FROM SALES;
```

106) Find lowest sale per store.

```
SELECT STORE_ID,SALE_ID,AMOUNT,FIRST_VALUE(AMOUNT)OVER(PARTITION BY STORE_ID
ORDER BY AMOUNT )AS LOW_SALE_PER_STORE FROM SALES
```

107) Compare category performance.

```
SELECT P.CATEGORY,SUM(SA.AMOUNT)AS REVENUE,
FIRST_VALUE(SUM(SA.AMOUNT))OVER (ORDER BY SUM(SA.AMOUNT)DESC) AS BEST_CATE
FROM SALES JOIN PRODUCT P ON SA.PRODUCT_ID = P.PRODUCT_ID GROUP BY P.CATEGORY;
```

108) Find best customer per store.

```
SELECT DISTINCT S.STORE_NAME,FIRST_VALUE(C.CUSTOMER_NAME)OVER(PARTITION BY
S.STORE_ID ORDER BY SA.AMOUNT DESC)AS BEST_CUSTOMER,
FIRST_VALUE(SA.AMOUNT) OVER (PARTITION BY S.STORE_ID ORDER BY SA.AMOUNT DESC)AS
MAX_SPENT FROM SALES SA
JOIN STORES S ON SA.STORE_ID = S.STORE_ID
JOIN CUSTOMERS C ON SA.CUSTOMER_ID = C.CUSTOMER_ID;
```

109) Find worst performing category.

```
SELECT DISTINCT FIRST_VALUE(P.CATEGORY)OVER(ORDER BY SA.AMOUNT ASC)AS
LOW_SALE_RECORD FROM SALES SA JOIN PRODUCTS P ON SA.PRODUCT_ID = P.PRODUCT_ID;
```

110) Analyze gap between best and worst.

```
SELECT AMOUNT, FIRST_VALUE(AMOUNT)OVER(ORDER BY AMOUNT DESC)-
FIRST_VALUE(AMOUNT)
OVER(ORDER BY AMOUNT)AS GAP FROM SALES;
```

SECTION 11

111) Find daily sales.

```
SELECT TRUNC(SALE_DATE)AS SALES_DAY,SUM(AMOUNT)AS DAILY_SALES
FROM SALES GROUP BY TRUNC(SALE_DATE) ORDER BY TRUNC(SALE_DATE);
```

112) Find monthly sales.

```
SELECT EXTRACT(MONTH FROM SALE  DATE)AS MONTHS,SUM(AMOUNT)AS MONTHLY  SALES
FROM SALES GROUP BY EXTRACT(MONTH FROM SALE  DATE)
ORDER BY MONTHS;
```

113) Find weekend sales.

```
SELECT SALE  DATE,SUM(amount)AS WEEKEND  SALES FROM SALES
WHERE TO_CHAR(SALE_DATE,'DY')IN('SAT','SUN') GROUP BY SALE_DATE ORDER BY SALE_DATE;
```

114) Find first sale date.

```
SELECT MIN(SALE_DATE)AS FIRST_SALE FROM SALES;
```

115) Find last sale date.

```
SELECT MAX(SALE_DATE)AS FIRST_SALE FROM SALES;
```

116) Find sales in specific month.

```
SELECT * FROM SALES WHERE SALE_DATE BETWEEN DATE '2025-01-01' AND DATE '2025-01-31';
```

117) Find sales growth month-wise.

```
SELECT EXTRACT(MONTH FROM SALE  DATE)AS MONTHS,SUM(AMOUNT)AS REVENUE,
SUM(AMOUNT) - LAG(SUM(AMOUNT)) OVER(ORDER BY EXTRACT(MONTH FROM SALE  DATE))AS
GROWTH
FROM SALES GROUP BY EXTRACT(MONTH FROM SALE  DATE)
ORDER BY MONTHS;
```

118) Find inactive customers (30 days).

```
SELECT DISTINCT CUSTOMER  ID FROM SALES WHERE sale  date
BETWEEN DATE '2025-03-01' AND DATE '2025-03-31';
```

119) Find repeat customers.

```
SELECT CUSTOMER_ID FROM SALES GROUP BY CUSTOMER_ID HAVING COUNT(*)>1;
```

120) Find sales trend.

```
SELECT SALE_DATE,SUM(AMOUNT) OVER (ORDER BY SALE_DATE
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW )AS RUNNING_SALE
FROM SALES;
```